# Proactive Incident Prevention System Using AI-Based IT Ticket Pattern Analysis

Ahilya Chauhan
Vellore Institute of Technology
Vellore, India

Rushikesh Nirmal
Vellore Institute of Technology
Vellore, India

Kishan Patel
Vellore Institute of Technology
Vellore, India

***Abstract*—IT service desks usually recognise a new fault only after tickets about it have piled up or a major incident has been declared. Yet the first evidence is often sitting in the queue hours earlier: a handful of user reports describing the same symptom in different words, spread across agents and categories. Published work on ticket analytics has concentrated on classifying, routing and linking individual tickets, while incident detection research has mostly relied on machine-generated monitoring data. This paper presents TIDE (Ticket-based Incident Detection for Early warning), a framework that detects emerging issues directly from human-written service desk tickets. TIDE encodes each incoming ticket with a sentence-embedding model, assigns it in a single pass to an evolving set of issue clusters, scores each cluster's growth against a seasonally adjusted Poisson baseline, and uses a gradient-boosted classifier to estimate whether a growing cluster will escalate into a ticket spike or major incident. A drift monitor is meant to trigger model retraining when the ticket distribution shifts. We also propose an evaluation protocol for the task that replays tickets in time order, injects synthetic emerging issues with known onset times, and reports detection lead time, false alarms per week, and the number of tickets that arrive in the window gained by early detection. Because annotated real data was not available, we evaluate on a simulated service desk of about 63,000 tickets over ten months, with noisy wording, harmless bursts and concept drift, over 10 random seeds. TIDE detected 83% of 200 test faults at 0.48 false alarms per week (F1 0.81), whereas burst detectors on agent categories, keywords, LDA topics and lexical clusters reached an F1 of at most 0.13. Lead times were modest: the median alert came one hour before reactive recognition, and 44% of detections came at least three hours earlier, mostly for slowly building faults. Ablations show that the escalation classifier is essential, while two components did not help as designed: seasonal adjustment of the burst baseline lowered accuracy, and the drift detector never fired. The results suggest that the service desk queue can support proactive problem management, and they identify what must be validated on real tickets.**

***Index Terms*—AIOps, IT service management, incident management, emerging issue detection, sentence embeddings, streaming clustering, concept drift, early warning systems**

## I. INTRODUCTION

Consider a certificate that expires on an internal load balancer at two in the morning. By nine, the service desk has received forty tickets. One user writes "VPN not connecting", another "portal says connection not private", a third "cannot open the HR app since morning". Each ticket lands with a different agent, under a different category, and is handled on its own. Someone eventually notices the pattern, a major incident is raised, and the actual fix takes ten minutes. The hours before that moment are where most of the cost sits: lost productivity, repeated diagnostic work, and in many contracts, breached service level agreements.

This is the everyday reality of reactive IT support. The ITIL framework separates *incident management*, which restores service, from *problem management*, which finds and removes the underlying cause so that incidents stop recurring [1]. In practice, problem management is often starved of time, and the link between a burst of loosely related tickets and a single cause is made by people, late. Tickets are a lagging signal in one sense, since a user has already been affected. But they are also the only signal for large parts of the estate that monitoring does not cover well: software-as-a-service applications, end-user devices, identity and access flows, and business applications whose failures show up as "the screen is blank" rather than as a metric crossing a threshold. Users act as thousands of distributed sensors, and the service desk queue is where their readings arrive.

Research on artificial intelligence for IT operations (AIOps) has grown quickly [2]–[5]. On the ticket side, most of this work treats tickets one at a time: predicting category, assignment group or severity [6]–[8], or linking a ticket to related incidents [9]–[11]. Work that does detect incidents early usually starts from machine-generated data such as logs, metrics and monitor-generated issue reports [12]–[14]. Commercial ITSM products advertise "emerging issue" or "potential issue" features that cluster incoming tickets over a sliding window [15], [16], but their methods and accuracy are not published, so they cannot be compared or reproduced. Studies that cluster historical tickets to find recurring incidents [17] are retrospective: they describe what happened rather than raising an alert while it is happening. Two questions that matter to a service desk manager are therefore largely unanswered in the literature: *how early* can an emerging issue be recognised from user tickets alone, and *at what cost* in false alarms, given that ticket vocabulary and volume change month to month.

This paper addresses that gap. We propose TIDE, a framework that watches the ticket stream continuously, groups semantically similar tickets into issue clusters as they arrive, and raises an alert when a cluster grows in a way that is likely to end in a ticket spike or major incident. We also propose an evaluation protocol for the task, since a standard one does not exist. The study is organised around four research questions:

- **RQ1:** Does streaming clustering over sentence embeddings group noisy service desk tickets into issue families more accurately than traditional representations such as TF-IDF and LDA topics?
- **RQ2:** How early, and with how many false alarms, can emerging issues be detected from tickets, compared with reactive practice and simpler burst detectors?
- **RQ3:** How does detection quality change under concept drift, and does a drift-aware update strategy preserve it?
- **RQ4:** How many tickets, and how much handling effort, fall inside the time window gained by early detection?

The main contributions are as follows.

1) A problem formulation for emerging-issue detection from human-written service desk tickets, with explicit definitions of an emerging issue, an alert, detection lead time and avoidable tickets.
2) TIDE, an open and reproducible framework that combines single-pass semantic clustering, a seasonally adjusted Poisson burst score and an escalation classifier, with a drift monitor that triggers retraining.
3) An evaluation protocol based on chronological replay and synthetic injection of emerging issues with known onset, which makes lead time measurable even when real incident labels are scarce.
4) An empirical study on a simulated service desk of about 63,000 tickets over ten months, repeated over 10 seeds, that compares TIDE with reactive and burst-detection baselines, analyses drift, estimates the operational benefit of earlier detection, and reports where the design fell short. The simulator and code are released.

The rest of the paper is organised as follows. Section II reviews related work. Section III defines the problem. Section IV describes TIDE. Section V explains the experimental design, and Section VI reports the results. Section VII discusses implications and Section VIII threats to validity. Section IX concludes.

## II. Background and Related Work

### A. Ticket Classification, Routing and Severity Prediction

Automating the handling of individual tickets is the most studied problem in ITSM analytics. Early systems combined text features with metadata to categorise tickets and support application management services [18]. Transformer models changed the baseline: Ticket-BERT fine-tuned a language model to label incident tickets and was deployed with an active-learning loop in Microsoft's incident management system [6]. Ahmed *et al.* reviewed IT incident risk prediction [19] and compared classifiers for predicting incident severity [7]. With large language models (LLMs), several studies compared zero-shot generative models with fine-tuned encoders; a benchmark of 22 generative LLMs on issue report classification found fine-tuned BERT-style models more consistent and cheaper to run [8]. These methods answer "what is this ticket and who should handle it". They do not ask whether a set of tickets, taken together, signals something new.

### B. Incident Linking and Correlation

A second line of work links related incidents so that engineers can treat them as one. LiDAR learns representations from incident text and historical link structure [9]. LinkCM links customer-reported issues to monitor-reported system incidents and reports large reductions in triage time [10]. DiLink adds service dependency graphs to textual features to link incidents across services [11]. Linking is pairwise and usually performed after incidents have been opened by monitors or engineers. Our setting differs in two ways: the inputs are user tickets rather than engineered incidents, and the goal is to decide *when* a group of tickets deserves attention, not only whether two items are related.

### C. Incident Detection and Failure Prediction

Failure prediction has a long history on system logs and metrics [14], and large cloud providers predict node failures from telemetry [12]. MID identifies incidents from multi-dimensional, monitor-generated issue reports by searching for attribute combinations whose counts deviate from normal [13]. More recently, LLMs have been used to recommend root causes and mitigation steps for cloud incidents [20], [21], and agent benchmarks now include IT operations tasks [22]. These approaches assume structured or machine-generated signals. Human-written tickets are shorter, noisier and far more varied in vocabulary, and the same fault can be described in dozens of ways.

### D. Emerging Topic and Burst Detection

Outside IT operations, emerging-topic detection has been studied in news and social media under the name topic detection and tracking [23]. Burst detection methods model a stream as switching between rate states [24] or monitor windowed counts at several time scales [25]; classical control charts such as CUSUM [26] and EWMA [27] flag shifts in a monitored rate. Neural topic models such as BERTopic cluster document embeddings and describe each cluster with class-based TF-IDF [28]. These building blocks are mature, but they have not been assembled and evaluated for service desk tickets, where counts per issue are small, volume follows strong weekly cycles, and the label that matters (did this become a major incident?) is rare.

### E. Concept Drift in Ticket Data

Ticket distributions change as systems are upgraded, teams reorganise and new applications are introduced. Concept drift is well documented for bug report triage [29] and incident routing [30], and a recent systematic review of triage research lists drift and cold-start among the main open challenges [31]. General drift adaptation strategies include sliding-window retraining and change detectors such as ADWIN [32], [33]. For emerging-issue detection, drift is not a nuisance to be removed: a genuinely new issue *is* a local distribution shift. The method must adapt to slow background change without absorbing the very novelty it is meant to flag.

### F. Research Gap

Table I summarises representative studies. Three observations follow. First, ticket-based research focuses on per-ticket tasks. Second, early detection research mostly uses machine-generated data. Third, where emerging-issue detection from tickets exists, it is either retrospective or proprietary, and lead time is not reported. To our knowledge, no published study offers a reproducible method and evaluation protocol for detecting emerging issues from human-written service desk tickets that measures lead time, false-alarm cost and robustness to drift. This paper aims to fill that gap. Public benchmarks are a related problem: most AIOps results are reported on private data [34], and public issue-tracker datasets [35] come from software projects rather than IT service desks.

## III. Problem Formulation

Let $\mathcal{D} = \{d_1, d_2, \ldots\}$ be a stream of tickets ordered by creation time, where each ticket $d_i = (t_i, x_i, \mathbf{m}_i)$ has a timestamp $t_i$, free text $x_i$ (short and long description) and metadata $\mathbf{m}_i$ such as priority, reporter, location, affected configuration item (CI) and assignment group. Time is divided into windows $w$ of length $\Delta$ (one hour in this study).

**Issue family.** An issue family $E \subset \mathcal{D}$ is the set of tickets caused by the same underlying fault. Its *onset* $t_0(E)$ is the creation time of its first ticket.

**Reactive recognition time.** The time $t_r(E)$ at which a conventional service desk would recognise $E$. For real events it is the time a major incident or problem record was opened. For injected events, where no such record exists, we use a volume proxy: the first window in which the family receives at least $q$ tickets, which is the point at which a spike becomes visible to staff watching queue dashboards.

**Alert.** A detector emits alerts $a = (t_a, k)$, each pointing to a cluster $k$ at time $t_a$. An alert *matches* $E$ if at least a fraction $\rho$ of the tickets in cluster $k$ at time $t_a$ belong to $E$ and $t_0(E) \le t_a \le t_r(E) + \delta$, where $\delta$ is a small tolerance for late detection. Alerts that match no event are false alarms.

**Lead time.** For a detected event, with $t_a^*(E)$ the time of its first matching alert,

$$L(E) = t_r(E) - t_a^*(E). \tag{1}$$

Positive values mean the detector was earlier than reactive practice.

**Avoidable tickets.** If responders need time $r$ to act on an alert, the tickets that arrive in the window gained by early detection are

$$\Delta N(E, r) = \left|\{d \in E : t_a^*(E) + r < t_d \le t_r(E) + r\}\right|. \tag{2}$$

$\Delta N$ is an upper bound on the tickets that earlier action could have prevented, because it assumes the fix takes effect as soon as responders act.

**Objective.** Maximise event recall and lead time subject to a false-alarm budget of $\beta$ alerts per week, a constraint that reflects how much attention a problem manager can realistically give.

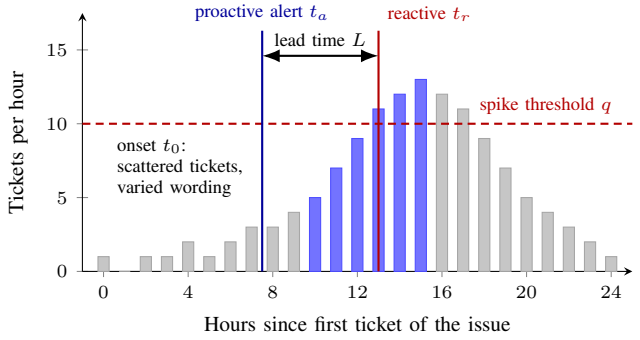



Fig. 1. Illustration of the quantities defined in Section III (synthetic counts, for explanation only). Blue bars are tickets arriving in the window gained by early detection for a response time $r = 2$ h, i.e., $\Delta N$ in (2).

Fig. 1 illustrates these quantities. The early part of an emerging issue is a small, slowly rising count of tickets that look unrelated on the surface. The detector's job is to see the shape early without firing on every random pair of similar tickets.

## IV. The TIDE Framework

### A. Overview

Fig. 2 shows the architecture. TIDE works in two stages. The representation stage cleans each ticket, encodes it as a dense vector and assigns it to an issue cluster in a single pass. The detection stage builds an hourly count series for every cluster, scores unusual growth, and passes candidate clusters to an escalation classifier that decides whether to alert. A drift monitor runs alongside both stages.

### B. Text Cleaning and Privacy Protection

Service desk text is messy. Tickets created from email carry signatures, disclaimers and reply chains; tickets from portals are often a single line. For each ticket we concatenate the short and long descriptions and apply rule-based cleaning: signature blocks (lines starting with "Regards", "Thanks", "Sent from" and similar) are cut, ticket references such as `RE: INC123456` and phone or extension numbers are removed, and leading greetings ("Hi team,", "Dear support,") are stripped. Signature removal also takes out most personal names. In a production deployment this step would be extended with URL, IP and host-name normalisation and a named-entity recogniser for personal data, applied before any text leaves the source system.

### C. Semantic Representation

Lexical representations struggle with tickets because the same fault is described with little word overlap ("Outlook keeps asking for password" versus "mail login popup again and again"). We therefore encode each cleaned ticket with a sentence-embedding model $f$ that maps text to a vector such that paraphrases lie close together. Transformer encoders such as Sentence-BERT [36] and its distilled MiniLM variants [37] are the usual choice. In this study we use WordLlama [38], a

TABLE I
COMPARISON OF REPRESENTATIVE RELATED WORK (✓: ADDRESSED; ∼: PARTLY ADDRESSED; ×: NOT A PRIMARY FOCUS)

| Study | Input data | Task | Emerging-issue detection | Lead-time evaluation | Drift handling |
|---|---|---|---|---|---|
| Ticket-BERT [6] | Incident tickets (cloud IcM) | Ticket labelling | × | × | ∼ |
| Ahmed *et al.* [7] | ITSM tickets | Severity prediction | × | × | × |
| Colavito *et al.* [8] | Software issue reports | Classification (LLM vs. BERT) | × | × | × |
| LiDAR [9] | Cloud incidents | Incident linking | × | × | × |
| LinkCM [10] | Customer + monitor incidents | Linking for triage | × | × | × |
| DiLink [11] | Machine-reported incidents, dependency graph | Incident linking | × | × | × |
| MID [13] | Monitor-generated issue reports | Incident identification | ✓ | × | × |
| Ferreira [30] | ITSM tickets | Routing under drift | × | × | ✓ |
| Al Fayed *et al.* [17] | ITSM tickets | Recurring-incident clustering | ∼ | × | × |
| Commercial tools [15], [16] | ITSM tickets | Emerging-issue correlation | ✓ | not published | not published |
| **TIDE (this work)** | **User-reported ITSM tickets** | **Emerging-issue detection and escalation** | ✓ | ✓ | ✓ |

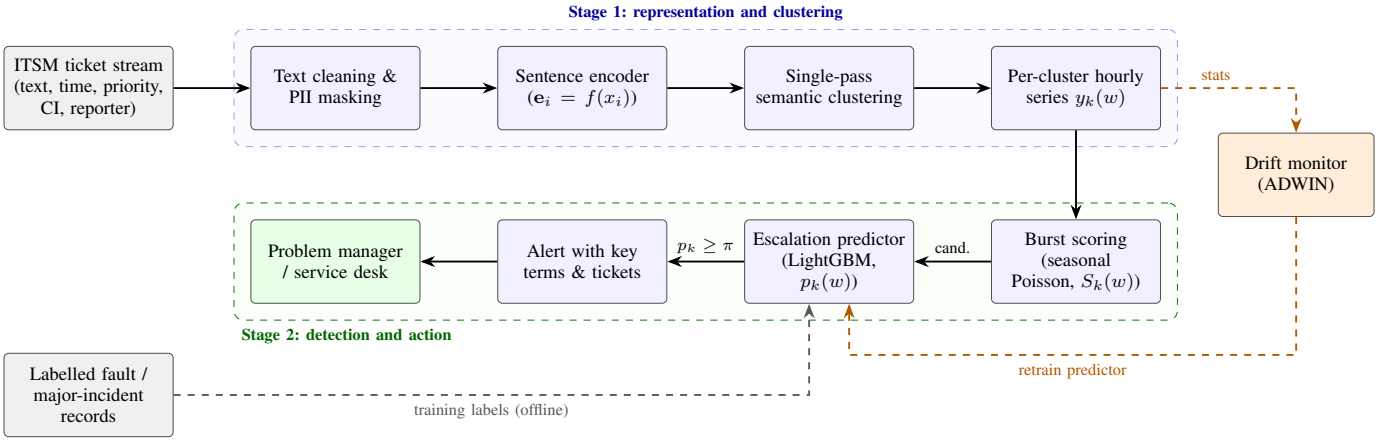



Fig. 2. Architecture of TIDE. Stage 1 turns each ticket into an embedding and assigns it to an issue cluster in one pass; Stage 2 scores cluster growth, predicts escalation and raises alerts. The drift monitor (dashed orange) watches daily clustering statistics and triggers retraining of the escalation predictor. Labelled fault records (major-incident records in a real deployment) are used only to build training labels.

256-dimensional embedding model built from recycled large-language-model token embeddings, because it runs on a CPU at tens of thousands of tickets per second and needs no GPU, which suits a desk-side deployment. Any sentence encoder can be substituted without changing the rest of the pipeline, and the encoder can be adapted to an organisation's vocabulary with unsupervised contrastive learning [39]. All vectors are $\ell_2$-normalised, so the dot product equals cosine similarity:

$$\mathbf{e}_i = \frac{f(x_i)}{\|f(x_i)\|_2}. \tag{3}$$

### D. Single-Pass Semantic Clustering

Batch clustering is a poor fit for a live queue, because re-clustering the full history every hour is expensive and changes cluster identities, which breaks the count series that detection depends on. TIDE instead maintains a set of active clusters, each with a unit centroid $\boldsymbol{\mu}_k$, a decayed weight $n_k$ and a last-update time. Fig. 3 shows the procedure for each new ticket.

The ticket is compared with every active centroid and its best match is

$$k^* = \arg\max_k \ \mathbf{e}_i^\top \boldsymbol{\mu}_k, \qquad s_i = \mathbf{e}_i^\top \boldsymbol{\mu}_{k^*}. \tag{4}$$

If $s_i \geq \tau$, the ticket joins cluster $k^*$ and the centroid is updated with exponential forgetting so that old tickets gradually lose influence:

$$n_{k^*} \leftarrow \gamma^{\,t_i - t_{k^*}}\, n_{k^*} + 1, \tag{5}$$

$$\boldsymbol{\mu}_{k^*} \leftarrow \frac{\left(n_{k^*} - 1\right)\boldsymbol{\mu}_{k^*} + \mathbf{e}_i}{\left\|\left(n_{k^*} - 1\right)\boldsymbol{\mu}_{k^*} + \mathbf{e}_i\right\|_2}, \tag{6}$$

where $\gamma \in (0, 1)$ is a per-hour decay factor and $t_{k^*}$ is the cluster's last update time. If $s_i < \tau$, the ticket opens a new cluster with centroid $\mathbf{e}_i$. With a few thousand active clusters, the nearest-centroid search is a single matrix-vector product; an approximate nearest-neighbour index can replace it at larger scale.

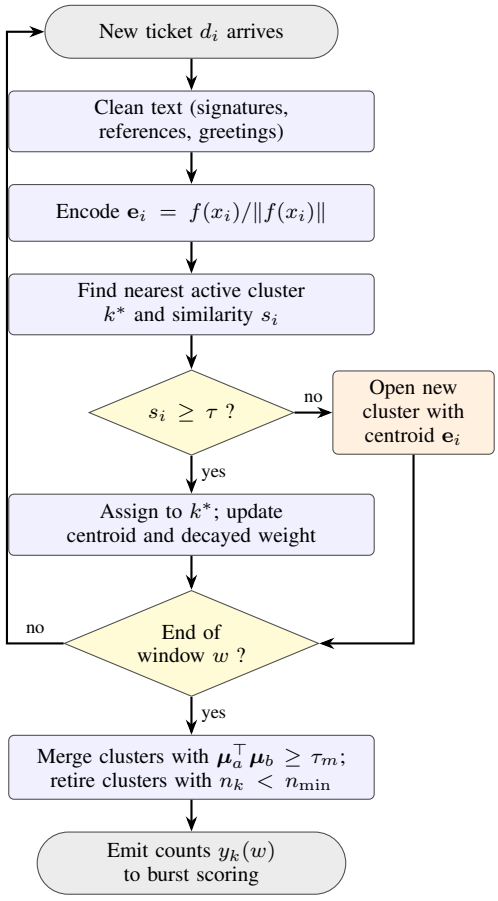



Fig. 3. Flowchart of the single-pass clustering step executed for every incoming ticket, with window-level merging and retirement.

Single-pass methods tend to fragment one issue into several clusters. At the end of each window, TIDE merges any two clusters whose centroids satisfy $\boldsymbol{\mu}_a^\top \boldsymbol{\mu}_b \geq \tau_m$ and retires clusters whose decayed weight falls below $n_{\min}$. When two clusters merge, their count histories and baselines are combined so that the series used for detection stays continuous. Retirement is checked once a day. Each cluster is described for humans by its most frequent content terms. A periodic batch consolidation with HDBSCAN [40], as used by BERTopic [28], is a natural extension but is not used in the experiments reported here.

### E. Burst Scoring

For each cluster, TIDE counts the tickets received in window $w$, giving a series $y_k(w)$. Counts per issue are small, often zero, one or two per hour, so Gaussian $z$-scores behave badly. We instead model the expected count as Poisson with a rate that combines the cluster's own recent level with the desk's weekly rhythm. Overall ticket volume swings sharply between a Monday morning and a Sunday night, so a raw count of five means different things at different hours. Let $h(w)$ be the ratio of the average total volume in the hour-of-week of $w$ to the overall hourly average, estimated from the training period and floored at 0.05. The cluster's deseasonalised level is tracked with an exponentially weighted moving average [27],

$$b_k(w) = \alpha \, \frac{y_k(w-1)}{h(w-1)} + (1-\alpha)\, b_k(w-1), \tag{7}$$

and the expected count for window $w$ is

$$\lambda_k(w) = \max\big(b_k(w)\, h(w),\ \lambda_{\min}\big), \tag{8}$$

where the floor $\lambda_{\min}$ gives new clusters a small prior rate. The burst score is the negative log tail probability of the observed count,

$$S_k(w) = -\log_{10} \Pr\big[Y \geq y_k(w)\big], \quad Y \sim \text{Poisson}\big(\lambda_k(w)\big). \tag{9}$$

A score of 3 means the count would be expected by chance about once in a thousand windows. A cluster becomes a *candidate* when $S_k(w) \geq \theta_S$ and $y_k(w) \geq y_{\min}$. The second condition stops a single ticket in a quiet hour from triggering anything.

### F. Escalation Prediction

Many bursts are harmless, for example a password-expiry wave on the first working day of a month, or a training session that produces a flurry of "how do I" questions. A burst score alone cannot tell these apart from the early phase of an outage. For each candidate, TIDE therefore computes the features in Table II and estimates the probability that the cluster is the early phase of a genuine fault that will escalate into an incident:

$$p_k(w) = g\big(\boldsymbol{\phi}_k(w)\big) \approx \Pr\big[\, k \text{ is a fault} \mid \boldsymbol{\phi}_k(w)\big). \tag{10}$$

In practice, a candidate is labelled positive for training when at least a fraction $\rho$ of its tickets in the last three hours belong to a fault that was later recorded as an incident (a major-incident or problem record in a real deployment; the injected fault in our simulation), and negative otherwise, so that harmless bursts become negative examples. The classifier $g$ is LightGBM [41] (300 trees, 15 leaves, learning rate 0.05), chosen because it handles mixed feature types, is fast to retrain and gives feature importances that problem managers can inspect. Because escalations are rare, we use balanced class weighting and select the alert threshold $\pi$ on the validation period so that the expected false-alarm rate stays within the budget $\beta$. An alert is suppressed if the same cluster raised one within a cool-down period $C$.

### G. Drift Monitoring and Adaptation

TIDE adapts at two speeds. Slow change is absorbed continuously by centroid forgetting and by the moving baseline $b_k(w)$. Larger shifts, such as the rollout of a new collaboration suite, require explicit action. The drift monitor tracks two daily signals: the share of incoming tickets that open a new cluster, and the mean nearest-centroid similarity $s_i$ of incoming tickets. Each signal is fed to an ADWIN change detector [33] ($\delta = 0.002$). When either detector fires (at most once every 14 days), TIDE retrains $g$ on the candidates of

TABLE II
FEATURES USED BY THE ESCALATION PREDICTOR

| Group | Features (computed per cluster $k$ at window $w$) |
|---|---|
| Volume | $y_k(w)$; burst score $S_k(w)$; expected rate $\lambda_k(w)$; growth ratio $y_k(w)/\lambda_k(w)$; counts over the last 3, 6 and 12 h; acceleration (second difference) |
| Spread | Distinct reporters, locations, CIs and agent-assigned categories among the cluster's tickets in the last 6 h |
| Severity | Share of P1/P2 tickets; share of tickets containing outage terms ("down", "error", "cannot", "everyone") |
| Novelty | Cluster age; cluster volume before the window; similarity of the cluster's founding ticket to its nearest existing cluster; mean nearest-centroid similarity of recent tickets |
| Context | Hour of day; day of week |

**Algorithm 1** TIDE online detection loop

**Require:** ticket stream $\mathcal{D}$; encoder $f$; thresholds $\tau, \tau_m, \theta_S, y_{\min}, \pi$; classifier $g$
$\mathcal{C} \leftarrow \emptyset$; $h(\cdot)$ and $g$ estimated on the training period
**for** each window $w$ **do**
  **for** each ticket $d_i$ created in $w$ **do**
    $\mathbf{e}_i \leftarrow \text{ENCODE}(\text{CLEAN}(x_i))$
    $k^* \leftarrow \arg\max_{k \in \mathcal{C}} \mathbf{e}_i^\top \boldsymbol{\mu}_k$
    **if** $\mathbf{e}_i^\top \boldsymbol{\mu}_{k^*} \geq \tau$ **then**
      add $d_i$ to $k^*$; update $\boldsymbol{\mu}_{k^*}, n_{k^*}$
    **else**
      create cluster with centroid $\mathbf{e}_i$; add to $\mathcal{C}$
    **end if**
  **end for**
  merge clusters with $\boldsymbol{\mu}_a^\top \boldsymbol{\mu}_b \geq \tau_m$; retire stale clusters
  **for** each cluster $k \in \mathcal{C}$ with new tickets in $w$ **do**
    compute $\lambda_k(w)$ and $S_k(w)$
    **if** $S_k(w) \geq \theta_S$ **and** $y_k(w) \geq y_{\min}$ **then**
      $p_k \leftarrow g(\boldsymbol{\phi}_k(w))$
      **if** $p_k \geq \pi$ **and** no alert for $k$ within $C$ **then**
        raise alert $(w, k)$ with key terms and tickets
      **end if**
    **end if**
    update $b_k(w)$
  **end for**
  at day end: update drift detectors; **if** drift **then** retrain $g$
**end for**

the most recent $M = 3$ months, whose labels are assumed to be available from analysts' confirmation or rejection of earlier alerts. A single emerging issue raises the new-cluster rate only briefly and in one region of the embedding space, whereas genuine drift persists across many clusters. In our design ADWIN therefore operates on daily aggregates, which keeps short incident bursts from being mistaken for drift.

### H. Alerting and Issue Summaries

An alert is useful only if a busy person can understand it in a few seconds. Each alert shows the cluster's key terms, its count series, the affected CIs and locations, and up to five representative tickets (the most central and the most recent). An LLM could additionally draft a short summary of the suspected issue, following recent work on LLM assistance for incident root-cause analysis [20], [21]; such summaries would be shown only as suggestions that cite ticket identifiers. LLM summaries are not part of the evaluated system in this paper. Algorithm 1 summarises the online loop.

## V. EXPERIMENTAL DESIGN

### A. Dataset: A Simulated Service Desk

Real service desk data with annotated major incidents is confidential and was not available for this study. We therefore built a ticket-stream simulator that reproduces the properties that make the task hard, and evaluate every method on it. The simulator and all code are released (Section VII) so that the same protocol can be run on real data. Table III summarises the resulting stream.

**Volume and seasonality.** Background tickets arrive as a Poisson process whose hourly rate follows a typical office profile: near zero at night, a peak of about 28 tickets per hour in the late morning, a lunch dip, and weekend volume of 8–15% of a weekday. The stream covers 301 days (about 10 months) for a simulated organisation of 3,000 employees in eight locations.

**Recurring issues and wording.** Background tickets come from 40 recurring issue families (password resets, VPN, Outlook, printers, SAP, Teams and so on) with Zipf-like frequencies. Each family has two to seven phrasing templates with interchangeable vocabulary slots (for example, "Outlook", "mail", "my mailbox"), and every ticket is wrapped in randomly chosen greetings, context phrases ("since morning", "after the update"), urgency phrases and e-mail signatures. Fifteen per cent of tickets contain a typo, 35% are lowercased and 8% carry a reply prefix with a ticket number. Agent-assigned categories are noisy: 18% of background tickets are filed under a random category.

**Harmless bursts.** Four kinds of benign burst produce volume spikes that are *not* faults: password-expiry waves on the first working day of each month, patch-day update tickets on the second Wednesday, fortnightly onboarding waves, and short "how do I" bursts after training sessions at a single site.

**Emerging faults.** Thirty fault scenarios (for example, single sign-on failure, an expired portal certificate, Citrix desktops not launching, a floor-wide printing failure) are injected as emerging issues. Their tickets use scenario-specific templates, so the same fault arrives in several wordings, and they are spread over three plausible categories (50/30/20%) plus 10% random categories, imitating the way agents file the same fault differently. Several scenarios overlap in vocabulary with background families (for example, a VPN gateway fault versus routine VPN tickets). Five scenarios are site-specific and affect only one location.

**Concept drift.** Thirty-five days into the test period the simulated organisation replaces its HR portal with Workday (changing the vocabulary of an existing family), retires Skype in favour of Teams (removing one family, increasing another and adding a new Teams Phone family), and grows its ticket volume by 25%. The launch of the new HR tool also produces a three-day burst of how-to tickets.

The stream is split chronologically into 151 days for training, 30 days for validation and threshold selection, and 120 days (about 17 weeks) for testing. No ticket from a later

TABLE III
SUMMARY OF THE SIMULATED TICKET STREAM (SEED 0; BACKGROUND IDENTICAL ACROSS SEEDS)

| Property | Value |
|---|---|
| Simulated period | 301 days (≈10 months) |
| Total tickets (mean over seeds) | 63,248 |
| background / benign-burst / fault (seed 0) | 58,354 / 1,355 / 3,582 |
| Tickets per day (mean / max) | 210 / 407 |
| Recurring issue families | 40 (+1 after drift) |
| Benign burst episodes | 66 |
| Fault scenarios (train-val / test pool per seed) | 15 / 15 |
| Injected faults per seed (train / val / test) | 30 / 7 / 20 |
| Mean tickets per fault | 62.2 |
| Distinct CIs / agent categories | 33 / 17 |
| Median ticket length after cleaning | 8 tokens |
| Train / validation / test (days) | 151 / 30 / 120 |
| Train / validation / test (tickets, seed 0) | 30,158 / 5,900 / 27,233 |
| Random seeds | 10 |

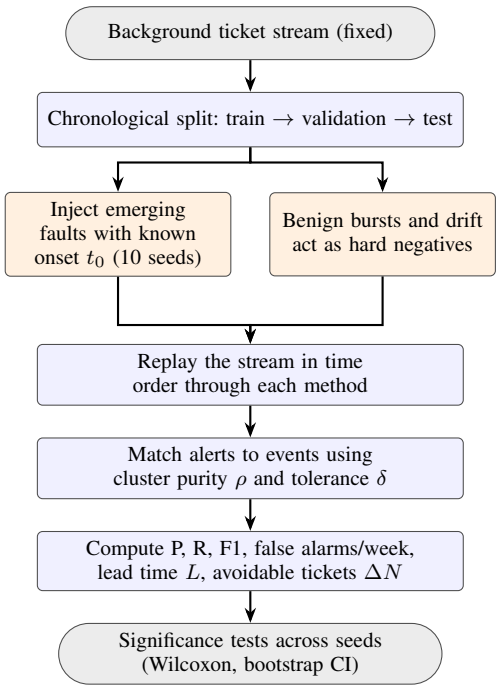



Fig. 4. Evaluation protocol. Injected faults give exact onset times for measuring lead time. With real data, labelled major incidents would be added to the injected events.

period is used to train or tune a model evaluated on an earlier one [42].

### B. Evaluation Protocol

Fig. 4 shows the protocol. Every method replays the full stream hour by hour in time order and may use only tickets that have already arrived.

**Injected faults.** For each of 10 random seeds, the 30 fault scenarios are split at random into 15 used for training and validation and 15 used only for testing, so test faults are never seen during training. Each seed injects 30 faults in the training period, up to 8 in the validation period and 20 in the test period, at random onset hours at least 72 hours apart. A fault's arrival rate follows a logistic curve modulated by the working-hours profile,

$$r(t) = \frac{A}{1 + \exp\big(-\kappa\,(t - t_{\text{mid}})\big)} \cdot \max\big(\tilde{h}(t), 0.12\big), \quad (11)$$

where $\tilde{h}(t)$ is the office profile scaled to a maximum of 1, the peak rate is $A \in \{6, 10, 16\}$ tickets per hour (60% of that for site-specific faults), and the growth setting is slow ($\kappa = 0.35$, midpoint 14 h after onset), medium (0.7, 7 h) or fast (2.0, 2 h). Because users mostly report problems during working hours, a fault that starts at night produces few tickets until the next morning, as in practice. The desk recognises the fault reactively at $t_r$, the end of the first hour with at least $q = 5$ of its tickets, and the fault is fixed $U(2, 6)$ hours later, after which its ticket rate halves every hour. Background tickets are generated with a fixed random state, so all seeds share the same background and differ only in the injected faults.

**Matching.** An alert at window $w$ is issued at $t_a = w + 1$ h. It matches fault $E$ if at least $\rho = 0.5$ of the tickets that the alerted group received in the last three hours belong to $E$ and $t_a \le t_r(E) + \delta$ with $\delta = 2$ h. Alerts on a fault after that window are counted as redundant (neither correct nor false), and all other alerts are false alarms, including alerts on benign bursts.

### C. Baselines

All baselines use the same seasonal Poisson burst score as TIDE on their own grouping of tickets, so that differences come from how tickets are grouped and filtered.

- **Reactive**: recognition at $t_r$, i.e., lead time zero by construction.
- **Category-EWMA**: burst scoring on hourly counts per agent-assigned category, approximating the category dashboards many desks use.
- **Keyword burst**: burst scoring on hourly counts of individual content terms (terms occurring in at least five training tickets), a single-level simplification of term burst detection [24].
- **LDA topics**: an LDA model [43] with 60 topics trained on the training period; each ticket is assigned to its most probable topic.
- **TF-IDF + online clustering**: the TIDE single-pass clustering applied to TF-IDF vectors, followed by burst scoring without the escalation predictor.

Ablations of TIDE remove the escalation predictor (burst score only), remove the metadata features (spread, priority and context), remove the seasonal profile ($h(w) = 1$), or replace the embeddings with TF-IDF vectors while keeping the full predictor.

### D. Metrics

**Clustering (RQ1).** The simulator records the true issue family or fault of every ticket. On a random sample of 10,000 test tickets per seed we report adjusted mutual information (AMI) [44], homogeneity and completeness against these labels, the number of clusters, and throughput. As a

batch reference we also cluster the sample's embeddings with HDBSCAN [40] (minimum cluster size 5, noise points as singletons).

**Detection (RQ2, RQ3).** We report event-level precision (correct alerts over correct plus false alerts), recall (detected test faults over all test faults), F1, false alarms per week, and lead time (1) pooled over all detected faults of all seeds. The area under the precision-recall curve (PR-AUC) [45] is computed at candidate level, scoring each candidate (group, hour) by the method's alert score. For RQ3, F1 is computed separately for each 30-day month of the test period.

**Operational benefit (RQ4).** We report $\Delta N$ from (2) for response times $r \in \{2, 4, 8\}$ hours. Effort is estimated with an assumed handling time of 15 minutes per ticket, since the simulator has no resolution times.

F1 differences between TIDE and every other method are tested with the two-sided Wilcoxon signed-rank test [46] over the 10 seeds, and a 95% confidence interval for the median lead time is obtained by bootstrap resampling of detected faults (5,000 resamples).

### *E. Implementation and Parameter Selection*

TIDE is implemented in Python 3.13 with WordLlama, scikit-learn, SciPy, LightGBM and the `river` library for ADWIN. The clustering threshold was chosen once, on the validation month of seed 0, by maximising AMI over $\tau \in \{0.45, 0.50, \ldots, 0.75\}$ for embeddings and $\{0.20, 0.25, \ldots, 0.50\}$ for TF-IDF, giving $\tau = 0.60$ and $\tau = 0.30$ respectively. Other fixed settings are $\tau_m = \tau + 0.1$, a centroid half-life of 7 days, $n_{\min} = 0.05$, $\alpha = 0.1$, $\lambda_{\min} = 0.1$ and a cool-down of $C = 12\,\text{h}$. Alert thresholds are tuned separately for every method and seed on the validation period: for methods with the escalation predictor, $\theta_S \in \{1.5, 2, 3\}$, $y_{\min} \in \{2, 3\}$ and $\pi \in \{0.05, 0.10, \ldots, 0.95\}$; for burst-only methods, $\theta_S \in \{2, 2.5, \ldots, 12\}$ and $y_{\min} \in \{2, 3, 4\}$ (wider grids for keywords). The configuration with the highest validation F1 among those with at most $\beta = 3$ false alarms per week is selected. All experiments ran on a 2-core cloud virtual machine with 7 GB of memory; one complete seed, including all baselines, takes about 1.6 minutes.

## VI. RESULTS AND DISCUSSION

All numbers below are means over the 10 seeds unless stated otherwise. Each seed's test period contains 20 injected faults, so detection results cover 200 test faults in total.

### *A. RQ1: Quality of Issue Clustering*

Table IV compares the ways of grouping tickets. Single-pass clustering of WordLlama embeddings reached an AMI of 0.75, clearly above LDA (0.64) and batch HDBSCAN on the same embeddings (0.54), and the same as single-pass clustering of TF-IDF vectors (0.75). The two streaming representations make different errors. Embedding clusters are purer (homogeneity 0.91 against 0.84) but split issue families more often (375 against 151 clusters in 10,000 tickets, completeness 0.69 against 0.71). HDBSCAN produced very pure but heavily fragmented clusters (3,109 clusters, homogeneity 0.98), because many short, idiosyncratically worded tickets end up as noise points. Both streaming methods clustered roughly 18,000 tickets per second on two CPU cores, excluding encoding, far above the peak arrival rate of a desk, which is a few dozen tickets per hour.

TABLE IV
RQ1: CLUSTERING QUALITY ON 10,000 TEST TICKETS PER SEED (MEAN OVER 10 SEEDS; S.D. SHOWN FOR AMI, ALL OTHERS $\leq 0.01$)

| Method | AMI | Hom. | Comp. | Clust. | Tickets/s |
|---|---|---|---|---|---|
| TF-IDF + online clust. | $0.75 \pm 0.01$ | 0.84 | 0.71 | 151 | 17,791 |
| LDA, 60 topics | $0.64 \pm 0.01$ | 0.67 | 0.64 | 60 | 2,852 |
| Batch HDBSCAN† | $0.54 \pm 0.01$ | 0.98 | 0.54 | 3,109 | 297 |
| **TIDE (WordLlama)** | $0.75 \pm 0.00$ | 0.91 | 0.69 | 375 | 18,791 |

†On WordLlama embeddings of the sample; noise points counted as singletons. Throughput excludes text encoding; LDA throughput includes model fitting.

The tie between embeddings and TF-IDF should be read with care. Simulated tickets are built from templates whose key nouns ("SAP", "VPN", "printer") recur across paraphrases, which favours lexical matching. Real tickets, with more free-form wording, would likely widen the gap in favour of embeddings, but this has to be tested on real data.

### *B. RQ2: Detection Accuracy and Lead Time*

Table V reports event-level detection. TIDE detected 83% of the 200 test faults with a precision of 0.80, an F1 of 0.81 ($\pm$0.07 across seeds) and 0.48 false alarms per week. All four baselines stayed below an F1 of 0.13, even though their thresholds were tuned on the same validation data and allowed up to three false alarms per week. TIDE's F1 was higher than every baseline's in all ten seeds (Wilcoxon $p = 0.002$, the smallest value attainable with ten pairs).

The weakest baseline, Category-EWMA (F1 0.05), illustrates the motivating problem. Because a fault's tickets are filed under several categories by different agents, no single category series rises far enough above its large background volume. Keyword and topic bursts suffer from the opposite problem: they fire on any frequent term or topic, including harmless bursts such as password-expiry waves. Single-pass TF-IDF clustering groups tickets well (RQ1), but without the escalation filter it reaches only an F1 of 0.12.

**Lead time.** Early warning turned out to be harder than detection. Over the 166 detected faults, the median lead over reactive recognition was 1.0 hour (95% bootstrap CI 0–3 hours; interquartile range 0–10.8 hours; mean 7.9 hours; maximum 66 hours). In 53% of detections TIDE alerted before the desk would have recognised the fault, and in 44% it did so by three hours or more; the remaining detections came in the same hour as reactive recognition or at most two hours after it. The reason is visible in the simulated arrival process and is, we believe, realistic: most faults produce few tickets until users start work, and then reach the spike threshold within an hour or two, which leaves little room to be early. Lead time was largest for slow-building faults (median 2.0 hours, recall 0.79), smaller for medium (1.0 hour, recall 0.83) and zero for

TABLE V
RQ2: EVENT-LEVEL DETECTION ON 200 TEST FAULTS (10 SEEDS). LEAD TIME IS THE MEDIAN OVER DETECTED FAULTS, IN HOURS; PR-AUC IS COMPUTED AT CANDIDATE LEVEL.

| Method | P | R | F1 | FA/wk | Lead | PR-AUC |
|---|---|---|---|---|---|---|
| Reactive (reference) | – | – | – | – | 0.0 | – |
| Category-EWMA | 0.04 | 0.09 | 0.05 | 2.76 | 0.0 | 0.05 |
| Keyword burst | 0.13 | 0.14 | 0.13 | 2.35 | 0.0 | 0.06 |
| LDA topics | 0.09 | 0.15 | 0.09 | 2.78 | 0.0 | 0.05 |
| TF-IDF + online clust. | 0.12 | 0.17 | 0.12 | 1.81 | 0.0 | 0.09 |
| **TIDE (full)** | **0.80** | **0.83** | **0.81** | **0.48** | **1.0** | **0.85** |
| *Ablations of* TIDE | | | | | | |
| w/o escalation predictor | 0.17 | 0.23 | 0.19 | 1.67 | −1.0 | 0.08 |
| w/o metadata features | 0.60 | 0.65 | 0.61 | 1.06 | 0.0 | 0.66 |
| w/o seasonal profile | 0.90 | 0.95 | 0.92 | 0.37 | 0.0 | 0.91 |
| TF-IDF instead of emb. | 0.78 | 0.72 | 0.74 | 0.41 | 4.0 | 0.79 |

fast faults (recall 0.87), whose first ticket preceded reactive recognition by a median of only 7.8 hours, most of it outside working hours. Site-specific faults, which affect one location and produce fewer tickets, were detected as reliably as global ones (recall 0.89 on 19 faults against 0.82 on 181).

**Ablations.** The escalation predictor is the component that makes TIDE work. Without it, the burst score alone reaches an F1 of 0.19 at 1.67 false alarms per week, no better than the baselines, because harmless bursts and genuine faults look alike in counts. Removing the metadata features lowers F1 to 0.61 ($p = 0.002$). By LightGBM gain, the most informative features were the share of P1/P2 tickets (35%), the cluster's volume before the current window (15%), cluster age (11%), hour of day (7%) and the similarity of the cluster's founding ticket to existing clusters (6%). The weight on priority partly reflects the simulator, in which fault tickets are more often P1 or P2 than background tickets; on real data, where users and agents set priority less consistently, this signal will be weaker.

Two ablations did not behave as we expected, and we report them as they are. First, *removing* the seasonal profile improved results (F1 0.92, 0.37 false alarms per week, $p = 0.002$). In the simulation, fault tickets follow the same working-hours rhythm as background tickets, so multiplying the expected rate by $h(w)$ raises the expectation in exactly the morning hours in which faults surface, damping their burst scores. Dividing night-time counts by a small $h(w)$ also inflates a cluster's baseline after a single late ticket. A profile that is applied only to the background share of a cluster, or none at all, therefore appears preferable; we keep the pre-specified design as the reported system rather than choosing components on test results. Second, replacing embeddings with TF-IDF lowered F1 to 0.74 ($p = 0.027$) but gave a longer median lead (4.0 hours) and fewer false alarms, so neither representation dominates on every measure in this setting.

**False alarms.** About half (51%) of TIDE's false alarms fell on the simulated harmless bursts (password-expiry waves, patch days, onboarding and training sessions, and the launch of the new HR tool); the rest were chance spikes in background clusters. At 0.48 alerts per week, a problem manager would review about eight false alarms over the 17-week test period.

TABLE VI
RQ3: F1 BY 30-DAY TEST MONTH (DRIFT BEGINS IN M2), MEAN FALSE ALARMS PER WEEK, AND NUMBER OF RETRAINING EVENTS PER SEED

| Strategy | M1 | M2 | M3 | M4 | Mean | FA/wk | Retr. |
|---|---|---|---|---|---|---|---|
| Static (trained once) | 0.89 | 0.67 | 0.82 | 0.87 | 0.82 | 0.48 | 0 |
| Periodic (monthly) | 0.89 | 0.59 | 0.78 | 0.91 | 0.80 | 0.75 | 3 |
| Drift-triggered (ADWIN) | 0.89 | 0.67 | 0.82 | 0.87 | 0.82 | 0.48 | 0 |

### C. RQ3: Robustness to Concept Drift

Table VI shows F1 by test month. The drift occurs on day 35 of the test period, inside month M2. With a predictor trained once, F1 fell from 0.89 in M1 to 0.67 in M2 and false alarms rose from 0.19 to 1.28 per week, mostly on the new Workday and Teams Phone clusters, which looked like novel, fast-growing issues. Without any retraining, F1 recovered to 0.82 and 0.87 in M3 and M4: once the new clusters had accumulated history, the volume and age features no longer marked them as novel, and centroid forgetting let the clustering absorb the changed vocabulary.

The two adaptation strategies did not help. The ADWIN detector on daily new-cluster rate and mean similarity never fired in any seed, so the drift-triggered strategy was identical to the static one. A daily aggregate that changes only gradually as old clusters age out was evidently too weak a signal at $\delta = 0.002$. Periodic monthly retraining on the last 90 days lowered mean F1 slightly (0.80 against 0.82) and raised false alarms in M2 (2.19 per week), most likely because the shorter window contains fewer positive examples than the full training period. Detecting and adapting to drift in ticket streams therefore remains an open problem in our design.

### D. RQ4: Estimated Operational Benefit

Table VII estimates how many tickets arrived in the window gained by early detection. Assuming responders act two hours after an alert, about 291 tickets per 17-week test period, or 23.5% of all fault tickets, arrived between the time action would have taken effect under TIDE and under reactive practice. At an assumed 15 minutes per ticket, this corresponds to roughly 73 analyst-hours. With slower responses (four or eight hours) the estimate rises to about 357 tickets, because the window shifts further into each fault's peak.

The benefit is concentrated. The median number of tickets gained per detected fault is only 3.5 (at $r = 2\,\text{h}$), against a mean of 17.5, because most of the total comes from the minority of slow faults that TIDE caught many hours early. These figures are upper bounds: they assume that a fix takes effect as soon as responders act and that an early alert is believed. Against them stands the cost of false alarms, about eight per test period, or under two hours of review at ten minutes each.

### E. Case Study

Fig. 5 follows one fault from seed 0: a slowly building SAP performance problem. Its first ticket arrived at 00:15 on a Sunday ("sap extremely slow today on my laptop") and was

TABLE VII
RQ4: TICKETS ARRIVING IN THE WINDOW GAINED BY EARLY DETECTION, PER 17-WEEK TEST PERIOD (MEAN OVER SEEDS)

| **Response time $r$** | **2 h** | **4 h** | **8 h** |
|---|---|---|---|
| Total $\Delta N$ (tickets) | 290.8 | 358.1 | 356.6 |
| Share of all test-fault tickets (%) | 23.5 | 29.0 | 28.8 |
| Mean $\Delta N$ per detected fault | 17.5 | 21.6 | 21.5 |
| Median $\Delta N$ per detected fault | 3.5 | 3.0 | 0.0 |
| Analyst effort (h, at 15 min/ticket) | 72.7 | 89.5 | 89.2 |

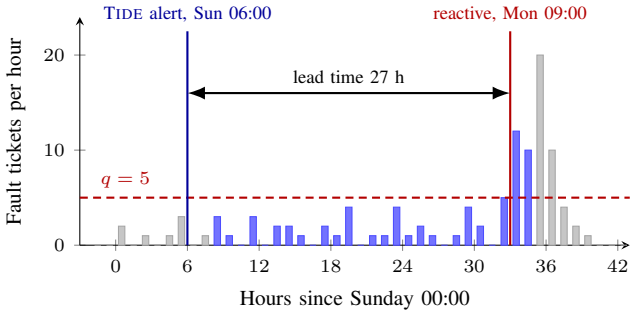



Fig. 5. Case study (seed 0): hourly tickets of a slowly building SAP fault. Each bar covers one hour. Blue bars are tickets in the gained window for a two-hour response, from $t_a + 2\,\text{h}$ to $t_r + 2\,\text{h}$.

filed under *Network*; the next two ("SAP GUI transactions hanging", "SAP GUI screen frozen for minutes") went to *Software* and *Business Apps*. Over the quiet weekend the fault produced zero to four tickets an hour. They joined TIDE's long-lived SAP cluster, which had collected about 1,200 routine SAP tickets over the previous months; all five tickets the cluster received between midnight and 06:00 on Sunday came from the fault (the other two fault tickets of that period were placed in other clusters). At 06:00, after seven fault tickets in total, the cluster passed the burst threshold and the predictor raised an alert, with the key terms "sap, gui, timing, out, slow". The desk would have recognised the problem reactively at 09:00 on Monday, when Monday-morning users pushed the count to five tickets in an hour. By then 49 fault tickets had arrived, spread over five categories (19 Business Apps, 16 Software, 12 Network, one Hardware and one General). The lead time was 27 hours, and with a two-hour response, 63 of the fault's 108 tickets fell in the gained window.

## VII. IMPLICATIONS AND LIMITATIONS

**For service desk and problem managers.** Early warning from tickets does not require new monitoring infrastructure; the data is already in the ticketing system. A detector like TIDE fits naturally into ITIL problem management: each alert is a candidate problem record, and its cluster provides the evidence that problem managers otherwise assemble by hand. Our results suggest setting expectations carefully. Grouping the right tickets and filtering out harmless bursts is achievable, and at under one false alarm per week the alerts are cheap to review. Large lead times, however, should be expected mainly for faults that build slowly or start outside working hours, as in the case study; for faults that hit many users at once, the main gains are the grouped evidence and the fact that the fault is recognised at all, not hours of warning.

**Tickets and monitoring are complementary.** Monitoring sees infrastructure; users see services. Many faults that reach the desk first, such as an expired certificate, a misconfigured group policy or a SaaS provider's regional outage, are invisible to internal metrics until users complain. Conversely, TIDE cannot help before the first user is affected. Combining ticket clusters with change records and monitoring alerts is a natural next step.

**What the simulation can and cannot show.** The simulator lets us measure lead time exactly and repeat the experiment ten times, which real data rarely allows. It also encodes our own assumptions: template-based wording, fault tickets with higher priority, and a reactive desk that recognises a fault as soon as five of its tickets arrive in one hour. The relative ordering of methods is therefore more trustworthy than the absolute values, and the seasonal-profile and priority findings in particular need to be checked on real tickets.

**Ethical considerations.** The evaluation uses only simulated tickets, with invented names and no personal data. In a real deployment, tickets contain personal and sometimes sensitive workplace information, so cleaning and anonymisation must run inside the organisation before analysis. The system flags issues, not individuals; its features describe clusters and are never used to profile reporters or agents.

**Data and code availability.** The simulator, the implementation of TIDE and all baselines, and the scripts that produce every table in this paper will be released at [repository URL]. Running them reproduces the reported numbers, and the README explains how to apply the same protocol to real ticket data.

## VIII. THREATS TO VALIDITY

**Internal validity.** The simulator and the detector were designed by the same authors, so the simulator may unintentionally favour the detector's assumptions. We reduced this risk by fixing the simulator before running the test evaluation, keeping test fault scenarios out of training, tuning every baseline with the same procedure and budget, and reporting results that went against our design (the seasonal profile and drift detection).

**Construct validity.** The reactive recognition time $t_r$ is a proxy. A real desk might notice a fault later than the first hour with five of its tickets, which would increase lead times, or earlier, through a phone call from an affected manager. The avoidable-ticket measure $\Delta N$ is an upper bound and should not be read as realised savings. Effort estimates rest on an assumed handling time of 15 minutes per ticket.

**External validity.** All results come from one simulated organisation with English, template-based tickets. Real desks differ in size, language, ticket culture and the way faults unfold. The protocol and code are released so that the study

can be replicated on real data; public issue-tracker data [35] could serve as an additional, if imperfect, test bed.

**Conclusion validity.** Results are averaged over ten seeds with 20 test faults each. Differences in F1 were tested with paired Wilcoxon tests, and the uncertainty of the median lead time is reported with a bootstrap interval. The background stream is the same in all seeds, so the seeds capture variation in faults but not in background traffic.

## IX. Conclusion and Future Work

This paper set out to test whether the service desk queue can be used for early warning rather than only for reactive repair. We formulated emerging-issue detection from human-written tickets as a streaming problem with explicit measures of lead time, false-alarm cost and avoidable tickets, and proposed TIDE, which combines single-pass semantic clustering, Poisson burst scoring, an escalation classifier and drift monitoring. We also introduced an evaluation protocol based on chronological replay and fault injection, which makes lead time measurable when real incident labels are scarce. On a simulated desk of about 63,000 tickets, TIDE detected 83% of emerging faults at 0.48 false alarms per week (F1 0.81), against an F1 of at most 0.13 for category, keyword, topic and lexical-cluster burst detectors. Lead times were modest (median one hour, at least three hours for 44% of detections) and concentrated in slowly building faults.

The study also points to what remains to be done. First, the seasonal adjustment and the drift detector both need redesign: removing the former improved accuracy, and the latter never fired. Second, the method must be evaluated on real service desk data with recorded major incidents, ideally in a prospective deployment in which problem managers act on live alerts, to measure realised rather than estimated benefits. Third, linking ticket clusters to change records and monitoring alerts could move detection earlier and point to likely causes. Finally, multilingual and code-mixed tickets, common in Indian workplaces, deserve dedicated study.

## Acknowledgment

The authors thank Vellore Institute of Technology, Vellore, for supporting this work.

## References

[1] AXELOS, *ITIL Foundation: ITIL 4 Edition*. London, U.K.: TSO, 2019.

[2] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-world challenges and research innovations," in *Proc. IEEE/ACM 41st Int. Conf. Software Engineering: Companion (ICSE-Companion)*, 2019.

[3] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOps methods for failure management," *ACM Trans. Intelligent Systems and Technology*, vol. 12, no. 6, 2021.

[4] Q. Cheng *et al.*, "AI for IT operations (AIOps) on cloud platforms: Reviews, opportunities and challenges," arXiv:2304.04661, 2023.

[5] Y. Remil, A. Bendimerad, R. Mathonat, and M. Kaytoue, "AIOps solutions for incident management: Technical guidelines and a comprehensive literature review," arXiv:2404.01363, 2024.

[6] Z. Liu, C. Benge, and S. Jiang, "Ticket-BERT: Labeling incident management tickets with language models," arXiv:2307.00108, 2023.

[7] S. Ahmed, M. Singh, B. Doherty *et al.*, "An empirical analysis of state-of-art classification models in an IT incident severity prediction framework," *Applied Sciences*, 2023.

[8] G. Colavito, F. Lanubile, and N. Novielli, "Benchmarking large language models for automated labeling: The case of issue report classification," *Information and Software Technology*, 2025, art. no. 107758, doi: 10.1016/j.infsof.2025.107758.

[9] Y. Chen, X. Yang, H. Dong, X. He, H. Zhang, Q. Lin, J. Chen, P. Zhao, Y. Kang, F. Gao, Z. Xu, and D. Zhang, "Identifying linked incidents in large-scale online service systems," in *Proc. 28th ACM Joint European Software Engineering Conf. and Symp. Foundations of Software Engineering (ESEC/FSE)*, 2020.

[10] J. Gu, J. Wen, Z. Wang, P. Zhao, C. Luo, Y. Kang, Y. Zhou, L. Yang, J. Sun, Z. Xu, B. Qiao, L. Li, Q. Lin, and D. Zhang, "Efficient customer incident triage via linking with system incidents," in *Proc. 28th ACM ESEC/FSE (Industry Track)*, 2020.

[11] S. Ghosh, K. Grover, J. Wong, C. Bansal, R. Namineni, M. Verma, and S. Rajmohan, "Dependency aware incident linking in large cloud systems," arXiv:2403.18639, 2024.

[12] Q. Lin *et al.*, "Predicting node failure in cloud service systems," in *Proc. 26th ACM ESEC/FSE*, 2018.

[13] J. Gu, C. Luo, S. Qin, B. Qiao, Q. Lin, H. Zhang, Z. Li, Y. Dang, S. Cai, W. Wu, Y. Zhou, M. Chintalapati, and D. Zhang, "Efficient incident identification from multi-dimensional issue reports via meta-heuristic search," in *Proc. 28th ACM ESEC/FSE*, 2020.

[14] F. Salfner, M. Lenk, and M. Malek, "A survey of online failure prediction methods," *ACM Computing Surveys*, vol. 42, no. 3, 2010.

[15] BMC Software, "How BMC Helix ITSM Insights identifies emerging issues," Product documentation, version 22.1. [Online]. Available: https://docs.helixops.ai, accessed: Oct. 2026.

[16] Aisera, "AI Ops: Potential issue identification," Product documentation. [Online]. Available: https://docs.aisera.com/ai-ops/potential-issue-identification, accessed: Oct. 2026.

[17] M. Al Fayed, E. Budiarto, and M. Galinium, "Identifying and analysing recurring incidents in IT service management using unsupervised machine learning: A case study from PT XYZ," Swiss German University, Tangerang, Indonesia, 2026.

[18] T.-H. Li, R. Liu, N. Sukaviriya, Y. Li, J. Yang, M. Sandin, and J. Lee, "Incident ticket analytics for IT application management services," in *Proc. IEEE Int. Conf. Services Computing (SCC)*, 2014.

[19] S. Ahmed, M. Singh, B. Doherty, E. Ramlan, K. Harkin, and D. Coyle, "AI for information technology operation (AIOps): A review of IT incident risk prediction," in *Proc. 9th Int. Conf. Soft Computing and Machine Intelligence (ISCMI)*, Toronto, Canada, 2022.

[20] T. Ahmed, S. Ghosh, C. Bansal, T. Zimmermann, X. Zhang, and S. Rajmohan, "Recommending root-cause and mitigation steps for cloud incidents using large language models," in *Proc. IEEE/ACM 45th Int. Conf. Software Engineering (ICSE)*, 2023.

[21] Y. Chen *et al.*, "Automatic root cause analysis via large language models for cloud incidents," in *Proc. 19th European Conf. Computer Systems (EuroSys)*, 2024.

[22] S. Jha, R. Arora, Y. Watanabe *et al.*, "ITBench: Evaluating AI agents across diverse real-world IT automation tasks," arXiv:2502.05352, 2025.

[23] J. Allan, J. Carbonell, G. Doddington, J. Yamron, and Y. Yang, "Topic detection and tracking pilot study: Final report," in *Proc. DARPA Broadcast News Transcription and Understanding Workshop*, 1998.

[24] J. Kleinberg, "Bursty and hierarchical structure in streams," *Data Mining and Knowledge Discovery*, vol. 7, no. 4, pp. 373–397, 2003.

[25] Y. Zhu and D. Shasha, "Efficient elastic burst detection in data streams," in *Proc. 9th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2003.

[26] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, 1954.

[27] S. W. Roberts, "Control chart tests based on geometric moving averages," *Technometrics*, vol. 1, no. 3, pp. 239–250, 1959.

[28] M. Grootendorst, "BERTopic: Neural topic modeling with a class-based TF-IDF procedure," arXiv:2203.05794, 2022.

[29] G. Chrupała, "Learning from evolving data streams: Online triage of bug reports," in *Proc. 13th Conf. European Chapter of the Association for Computational Linguistics (EACL)*, 2012.

[30] M. C. Ferreira, "Incident routing: Text classification, feature selection, imbalanced datasets, and concept drift in incident ticket management," Master's thesis, COPPE, Universidade Federal do Rio de Janeiro, Rio de Janeiro, Brazil, 2017.

[31] Y. Zhao, S. Zhang, Y. Wu, Y. Sun, Y. Sun, D. Pei, C. Bansal, and M. Ma, "Triage in software engineering: A systematic review of research and practice," arXiv:2511.08607, 2025.

[32] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, 2014.

[33] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proc. SIAM Int. Conf. Data Mining (SDM)*, 2007.

[34] Z. Li *et al.*, "Constructing large-scale real-world benchmark datasets for AIOps," arXiv:2208.03938, 2022.

[35] L. Montgomery, C. Lüders, and W. Maalej, "An alternative issue tracking dataset of public Jira repositories," in *Proc. 19th Int. Conf. Mining Software Repositories (MSR)*, 2022.

[36] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, 2019, pp. 3982–3992.

[37] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[38] D. L. Miller, "WordLlama: Recycled token embeddings from large language models," GitHub repository. [Online]. Available: https://github.com/dleemiller/WordLlama, 2024.

[39] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2021.

[40] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Proc. Pacific-Asia Conf. Knowledge Discovery and Data Mining (PAKDD)*, 2013, pp. 160–172.

[41] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[42] V. Cerqueira, L. Torgo, and I. Mozetič, "Evaluating time series forecasting models: An empirical study on performance estimation methods," *Machine Learning*, vol. 109, pp. 1997–2028, 2020.

[43] D. M. Blei, A. Y. Ng, and M. I. Jordan, "Latent Dirichlet allocation," *J. Machine Learning Research*, vol. 3, pp. 993–1022, 2003.

[44] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance," *J. Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.

[45] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[46] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.